

# Resumo para Prova — Python OO

Listas • Tuplas • Orientação a Objetos • Encapsulamento • Herança

---

## Aula 1 — Listas

Lista é uma coleção mutável de valores. Pode misturar tipos, cresce e diminui livremente.

### Criação e acesso:

```
numeros = [10, 20, 30]
print(numeros[0]) # 10 – primeiro
print(numeros[-1]) # 30 – ultimo
print(len(numeros)) # 3
```

### Modificando:

```
numeros.append(40) # adiciona no final
numeros.insert(1, 99) # insere na posicao 1
numeros.remove(20) # remove pelo valor
numeros.pop() # remove o ultimo
numeros.pop(0) # remove pelo indice
```

### Percorrendo:

```
for n in numeros: # jeito Python (preferido)
    print(n)

for i, n in enumerate(numeros): # quando precisar do indice E valor
    print(i, n)
```

Dica de prova: `IndexError` ocorre ao acessar índice fora do intervalo. Sempre cheque com `len()`.

---

## Aula 2 — Tuplas

Tupla é igual a lista, mas IMUTÁVEL — depois de criada não pode ser alterada. Usa parênteses.

### Criação e acesso:

```
meses = ('jan', 'fev', 'mar')
print(meses[0]) # jan
print(meses[-1]) # mar
print(len(meses)) # 3
```

### Metodos disponiveis:

```
numeros = (1, 2, 2, 3, 2)
print(numeros.count(2)) # 3 – conta ocorrencias
print(numeros.index(3)) # 3 – posicao do valor
```

### Tupla com 1 elemento (a virgula e obrigatoria):

```
certo = (42,) # tupla
errado = (42) # apenas o numero 42
```

| Use Lista quando...        | Use Tupla quando...         |
|----------------------------|-----------------------------|
| Precisa adicionar/remover  | Dados são fixos e não mudam |
| Notas, carrinho de compras | Coordenadas, dias da semana |

Dica de prova: tupla NAO tem append, remove nem insert — gera AttributeError.

---

## Aula 3 — Orientacao a Objetos (OO)

Organiza o codigo modelando o mundo real. Tudo tem características (atributos) e comportamentos (metodos).

### Classe e Objeto:

```
class Aluno: # CLASSE = molde / planta baixa
    def __init__(self, nome, nota): # construtor, executado ao criar
        self.nome = nome # atributo
        self.nota = nota

    def situacao(self): # metodo
        if self.nota >= 6:
            return 'Aprovado'
        else:
            return 'Reprovado'

aluno1 = Aluno('Ana', 8.5) # OBJETO = instancia da classe
print(aluno1.nome) # Ana
print(aluno1.situacao()) # Aprovado
```

| Conceito   | Em Python           | O que e                      |
|------------|---------------------|------------------------------|
| Classe     | class Aluno:        | Molde / planta baixa         |
| Objeto     | aluno1 = Aluno(...) | Instancia criada da classe   |
| Atributo   | self.nome           | Caracteristica do objeto     |
| Metodo     | def situacao(self): | Comportamento do objeto      |
| Construtor | def __init__(self): | Executado ao criar o objeto  |
| self       | self.nome = nome    | Referencia ao proprio objeto |

## Aula 4 — Encapsulamento

Protege os dados do objeto. Ninguem de fora mexe direto nos atributos — so atraves de metodos controlados.

### Atributos privados com \_\_ (duplo underline):

```
class Aluno:
    def __init__(self, nome, nota):
        self.__nome = nome # privado
        self.__nota = nota # privado

    def get_nota(self): # getter — le o valor
        return self.__nota

    def set_nota(self, nota): # setter — modifica com validacao
        if 0 <= nota <= 10:
            self.__nota = nota
        else:
            print('Erro: nota invalida')

aluno1 = Aluno('Ana', 8.5)
```

```
aluno1.set_nota(-50) # Erro: nota invalida
aluno1.set_nota(9.0) # permitido
print(aluno1.get_nota()) # 9.0
```

| Conceito         | Em Python             | O que faz                  |
|------------------|-----------------------|----------------------------|
| Atributo publico | self.nome             | Qualquer um acessa         |
| Atributo privado | self.__nome           | So a propria classe acessa |
| Getter           | def get_nome(self)    | Le o atributo              |
| Setter           | def set_nota(self, v) | Modifica com validacao     |

Dica de prova: acessar aluno1.\_\_nota de fora gera AttributeError. Use sempre o getter.

## Aula 5 — Heranca

Heranca permite que uma classe filha aproveite atributos e metodos da classe mae, evitando repeticao.

### Sintaxe basica:

```
class Pessoa: # classe MAE
def __init__(self, nome, idade):
self.nome = nome
self.idade = idade

def apresentar(self):
print(self.nome, self.idade)

class Aluno(Pessoa): # classe FILHA
def __init__(self, nome, idade, nota):
super().__init__(nome, idade) # chama o __init__ da mae
self.nota = nota # atributo proprio

def situacao(self): # metodo proprio
if self.nota >= 6:
return 'Aprovado'
return 'Reprovado'

aluno = Aluno('Ana', 17, 8.5)
aluno.apresentar() # herdado da mae: Ana 17
print(aluno.situacao()) # proprio da filha: Aprovado
```

| Conceito       | Em Python            | O que faz                   |
|----------------|----------------------|-----------------------------|
| Classe mae     | class Pessoa:        | Fornece atributos e metodos |
| Classe filha   | class Aluno(Pessoa): | Herda da classe mae         |
| super()        | super().__init__()   | Chama o construtor da mae   |
| Metodo herdado | aluno.apresentar()   | Vem da mae automaticamente  |
| Metodo proprio | def situacao(self):  | Existe so na filha          |

Dica de prova: sem super().\_\_init\_\_() a classe filha nao herda os atributos da mae.